

Technical Requirements & Product Specification Document

Project: *WorkGreg Web & Hybrid Desktop — Aggregator & Automated Job Finder*

Version: 2.1.0

Date: 19 September 2026

Team: Arthur & Orest

Table of Contents

- 1. Project Overview** 3
- 2. Architecture** 3
 - 2.1. High-Level Topology 4
 - 2.2. Component Breakdown 4
 - 2.3. Data Flow: Ingestion to Search 4
 - 2.4. Key Architectural Decisions 5
- 3. Frontend Views** 5
 - 3.1. Pages 5
 - 3.2. Components 6
- 4. Backend Endpoints** 6
 - 4.1. Authentication Endpoints (/api/v1/auth) 6
 - 4.2. Vacancies & Job Discovery Endpoints (/api/v1/jobs) 7
 - 4.3. User Profile & Saved Items Endpoints (/api/v1/user) 7
 - 4.4. Scraper & Background Task Endpoints (/api/v1/admin/scrapers) 7
- 5. Core Functional Requirements & User Stories** 7
 - 5.1. Function 1: Automated Vacancy Scraping & Ingestion 7
 - 5.2. Function 2: Advanced Search & Dynamic Filtering 8
 - 5.3. Function 3: Authentication & Profile Management 8
- 6. Feature Specifications & Requirements Matrix** 8
- 7. Primary Use Cases** 10
 - 7.1. UC-01: Server-Side Automated Scraping & Summarization 10
 - 7.2. UC-02: Natural Language AI Vacancy Search 10
 - 7.3. UC-03: Cloud Sync of Filter Presets 10

1. Project Overview

WorkGreg is a job aggregation and intelligence platform that brings listings from multiple job boards into a single, centralized workspace. Instead of checking several sites by hand, users search, filter, and semantically query one unified feed — with AI doing the work of summarizing postings and surfacing the ones that actually match what they're looking for.

Why it exists

Job hunting today means repeating the same search across many separate platforms, each with inconsistent filtering and no way to query by meaning rather than exact keywords. WorkGreg removes that duplication: one login, one search experience, and AI-assisted triage so users spend their time evaluating good-fit roles instead of scanning noise.

Target users

- Active job seekers who track roles across several job boards at once and want a single place to search, filter, and save.
- Passive candidates who want natural-language, semantic search (“senior backend roles with remote flexibility”) rather than rigid keyword filters.
- Power users / Pro subscribers who rely on saved presets, AI summaries, and cross-device sync to run an ongoing, low-effort search.

Project goals

- Aggregate postings from multiple job boards into one deduplicated, continuously refreshed feed.
- Reduce time-to-evaluate per posting using AI-generated 3-bullet summaries and stack/red-flag tagging.
- Support natural-language search via semantic vector matching, in addition to conventional filters.
- Provide a lightweight account system so filters, saved jobs, and presets follow the user across web and desktop.
- Ship web-first, with the same frontend packageable as a desktop app via Tauri, avoiding a second codebase.

Scope at a glance

The sections that follow cover the system architecture, the frontend views (pages and components), backend API surface, core functional requirements with user stories, a feature-by-feature requirements matrix, and the primary end-to-end use cases.

2. Architecture

WorkGreg is a web-first, single-page application backed by a centralized API service. The frontend is deliberately decoupled from any desktop packaging concern: the same Angular build that runs in a

browser is wrapped in Tauri v2 when a native desktop installer is needed, so there is exactly one frontend codebase to maintain.

2.1. High-Level Topology

A single Angular SPA talks to a centralized FastAPI service over a REST API secured with JWTs. The API service is the only component with direct database access; background workers populate that database asynchronously via scraping and AI pipelines, so read paths served to users stay fast regardless of scraping load.

2.2. Component Breakdown

Frontend — Angular 19+ SPA

- Web-first single-page application; deployable directly to cloud web hosting.
- Optionally packaged inside a Tauri v2 shell for a native desktop installer, from the same build artifact.
- Communicates exclusively over REST/JSON with the FastAPI backend, authenticated via a bearer JWT.

Backend — FastAPI (Python 3.10+)

- Centralized cloud service hosted on a Linux server/container cluster; stateless request handling so it scales horizontally.
- Owns all business logic: auth, job search/filtering, semantic search orchestration, and preset/bookmark management.
- Dispatches and coordinates background scraping and AI summarization work rather than performing it inline on the request path.

Database & Storage

- Hosted PostgreSQL for relational data and user state, extended with pgvector to store and query AI-generated embeddings alongside normal rows.
- Redis for scraping queues and task scheduling, decoupling background ingestion from the request/response cycle.

Authentication

- Lightweight OAuth2 with JWT access/refresh tokens, implemented natively in FastAPI or delegated to a managed identity provider.
- Role claims (Free / Pro / Admin) travel inside the JWT payload and gate access to higher-cost operations such as semantic vector search.

2.3. Data Flow: Ingestion to Search

- Playwright-based background workers scrape configured job boards on a schedule driven by a server-side cron/queue (Redis-backed).

- Each listing is hashed by URL before insertion so PostgreSQL rejects duplicate postings across sources.
- New listings pass through an LLM pipeline that produces a 3-bullet summary, stack tags, and an embedding vector, all persisted alongside the row.
- At query time, keyword filters run as indexed SQL, while natural-language queries are embedded and matched via pgvector cosine similarity — both paths return through the same jobs API.

2.4. Key Architectural Decisions

- Single frontend codebase: Tauri wraps the web build rather than maintaining a separate desktop client, keeping web and desktop in lockstep.
- Server-side scraping over local scripts: centralizing Playwright workers on the server means fresher, deduplicated data for every user without any local setup.
- In-database vector search: pgvector keeps semantic search co-located with relational data, avoiding a separate vector database and its sync overhead.
- Stateless JWT auth: no server-side session store is required, simplifying horizontal scaling of the API tier.

3. Frontend Views

The frontend is organized into full-page views routed by the Angular router, built from a smaller set of shared, reusable components. Pages compose components; components carry no page-specific routing logic of their own.

3.1. Pages

3.1.1. Authentication Pages

- Login View: Form for email and password, options for “Remember Me”, link to password reset, and submit button returning a JWT token.
- Register View: Form collecting full name, email, password, and account tier selection (Free or Pro).
- Password Reset View: Email submission screen to request password reset links.

3.1.2. Main Dashboard & Job Feed View

The primary landing view after login; composes the header, filter sidebar, and job feed grid components described in Section 3.2.

3.1.3. AI Semantic Vector Search View

- Conceptual Query Input: Prominent text prompt box accepting full sentences describing ideal roles or projects.
- Similarity Threshold Slider: User control to tweak vector distance match precision.

- Search Results List: Cards ranked by semantic match score with highlighting of matching context.

3.1.4. User Profile & Saved Items View

- Saved Jobs Tab: Table or card grid of user-bookmarked roles with application status trackers.
- Filter Presets Manager: List of saved filter configurations with options to load, rename, edit, or delete presets.
- Account Settings: Password updates, notification preferences, API keys, and subscription plan management.

3.1.5. Admin Dashboard View

Admin role only.

- Scraping Status Panel: Operational status of Playwright background scrapers, last execution timestamps, and error logs per platform.
- AI Usage Metrics: Monitoring LLM token consumption and embedding generation stats.
- User Management Table: User account list, assigned roles, and access control toggles.

3.2. Components

Shared, reusable building blocks used across the pages above.

3.2.1. Navigation & Layout Components

- Header Navigation Bar: User profile status, active account tier badge, saved presets quick-dropdown, search bar, and navigation tabs.
- Filter Sidebar: Collapsible side panel with controls for job board sources, remote status toggles, salary range sliders, experience level checkboxes, and tech stack tags.

3.2.2. Job Discovery Components

- Job Results Feed Grid: Main section displaying job cards with key details such as title, company logo, location, compensation, posting age, and platform origin tag.
- AI Key Takeaways Modal / Drawer: Expanding side drawer or overlay showing the 3-bullet LLM summary, core tech stack tags, identified red/green flags, and direct apply link.

4. Backend Endpoints

4.1. Authentication Endpoints (/api/v1/auth)

- **POST** /api/v1/auth/register — Registers a new user and returns user details.
- **POST** /api/v1/auth/login — Authenticates credentials and returns JWT access and refresh tokens.
- **POST** /api/v1/auth/refresh — Takes refresh token and issues a new access token.

- **GET** `/api/v1/auth/me` — Retrieves the currently authenticated user profile and permissions.

4.2. Vacancies & Job Discovery Endpoints (`/api/v1/jobs`)

- **GET** `/api/v1/jobs` — Fetches paginated job listings with parameters for sources, remote flag, min salary, and skill tags.
- **GET** `/api/v1/jobs/{job_id}` — Fetches full details for a single job listing including raw text and metadata.
- **POST** `/api/v1/jobs/vector-search` — Accepts a natural language query string, generates embeddings, executes pgvector similarity search, and returns ranked vacancies.
- **POST** `/api/v1/jobs/{job_id}/summarize` — Triggers on-demand LLM summary generation for a specific vacancy.

4.3. User Profile & Saved Items Endpoints (`/api/v1/user`)

- **GET** `/api/v1/user/saved-jobs` — Retrieves list of saved job vacancies for the logged-in user.
- **POST** `/api/v1/user/saved-jobs/{job_id}` — Bookmarks a job vacancy to the user account.
- **DELETE** `/api/v1/user/saved-jobs/{job_id}` — Removes a saved job from the user account.
- **GET** `/api/v1/user/presets` — Fetches all saved filter configurations for the authenticated user.
- **POST** `/api/v1/user/presets` — Saves a new filter configuration preset.
- **DELETE** `/api/v1/user/presets/{preset_id}` — Deletes a saved filter preset.

4.4. Scraper & Background Task Endpoints (`/api/v1/admin/scrapers`)

- **GET** `/api/v1/admin/scrapers/status` — Returns status and logs of all configured Playwright scrapers.
- **POST** `/api/v1/admin/scrapers/trigger` — Manually triggers a scraping job for a targeted platform source.

5. Core Functional Requirements & User Stories

5.1. Function 1: Automated Vacancy Scraping & Ingestion

User Story: As a job seeker, I want the system to continuously collect and summarize job postings from multiple job boards on a central server so that I always have access to fresh, deduplicated listings without running local scripts.

System Scope

- Server-side distributed browser automation using Playwright and background worker queues (e.g., Celery / ARQ).

- Centralized PostgreSQL database layer with URL hashing to prevent duplicate job entries across sources.
- Server-side cron scheduler executing routine scraping jobs at configurable intervals.
- LLM-driven parsing engine running on the server to structure raw job text and generate 3-bullet summaries.

5.2. Function 2: Advanced Search & Dynamic Filtering

User Story: As a user, I want to filter and semantically query job listings using specific attributes or natural language so that I can instantly zero in on relevant opportunities.

System Scope

- Multi-source toggles allowing users to filter listings by target platforms.
- Dynamic multi-parameter filter engine supporting salary, remote flags, tech tags, and location.
- Server-side Semantic Vector Search utilizing pgvector embeddings for natural language queries (e.g., *“Find Senior Python roles offering remote work with focus on backend microservices”*).

5.3. Function 3: Authentication & Profile Management

User Story: As a user, I want a lightweight account to store my saved vacancies, custom filters, and search preferences across all my devices.

System Scope

- Stateless JWT authentication (OAuth2 Password Bearer flow) managed natively by FastAPI.
- Role-based authorization (RBAC) to gate high-cost AI operations (e.g., semantic vector queries) based on user tiers.
- Cloud profile sync persisting user preferences, saved filters, and bookmarked listings across web sessions and optional desktop wrappers.

6. Feature Specifications & Requirements Matrix

Feature ID	Feature Area	Description & Technical Requirements	Acceptance Criteria
F-1.1	Scraping Engine	Server Playwright Worker: Headless browser automation running on server workers to parse dynamic SPAs.	Successfully extracts job title, company, salary, location, and raw text into PostgreSQL.
F-1.2	Data Persistence	Central PostgreSQL Store: Centralized DB storage using URL hash indexes to prevent duplicate listings.	Database rejects duplicate job postings and serves queries in < 100ms.

Feature ID	Feature Area	Description & Technical Requirements	Acceptance Criteria
F-1.3	Task Scheduler	Server Task Queue: Server-side cron/worker schedule executing automated fetch jobs.	Background tasks execute reliably without manual user triggers.
F-1.4	AI Processing	Server AI Summarizer: LLM pipeline extracting key stack requirements and generating bullet summaries.	Summaries generate and store in DB within < 2s per vacancy.
F-2.1	Source Control	Source Filter: UI toggle controls to select active target platforms.	Backend queries filter out unselected platforms instantaneously.
F-2.2	Filtering Engine	Dynamic Search: Indexed SQL queries filtering salary, location, tech stack, and posting age.	Filtered results return in < 50ms over REST API.
F-2.3	Semantic Search	pgvector AI Search: Vector similarity search calculated directly inside PostgreSQL.	Returns relevant vacancies matching natural language queries even without exact keyword matches.
F-3.1	Auth System	Lightweight JWT Auth: FastAPI OAuth2 Bearer tokens with secure password hashing (bcrypt).	Users sign up and log in securely via Web and Desktop UI without external redirects.
F-3.2	Authorization	User Tiering: Simple RBAC (Free, Pro, Admin) stored in the JWT payload.	REST API blocks Free tier accounts from executing expensive AI vector searches.
F-3.3	User State	Cloud Filter Presets: User-defined search criteria saved directly to the central user DB.	Filter presets sync automatically between web logins and desktop app launches.

7. Primary Use Cases

7.1. Use Case UC-01: Server-Side Automated Scraping & Summarization

- **Primary Actor:** Background Worker / System Cron
- **Preconditions:** Server worker pool active; Playwright instances running in containerized environment.

Main Success Scenario

1. The server scheduler triggers a scheduled scraping job for configured job boards.
2. Playwright worker instances fetch raw listing data and send payloads to FastAPI backend.
3. FastAPI checks the SHA-256 hash of the listing URL against PostgreSQL to ignore duplicates.
4. New entries pass through the LLM summarization pipeline to extract 3 key bullets and stack tags.
5. Processed vacancies are indexed into PostgreSQL (including vector embeddings).

7.2. Use Case UC-02: Natural Language AI Vacancy Search

- **Primary Actor:** Authenticated User (Pro Role)
- **Preconditions:** User is authenticated via JWT; job database populated.

Main Success Scenario

1. User enters a query into the web search bar (e.g., "Senior Angular developer with Tauri or Electron experience").
2. Angular frontend sends an HTTP GET request with JWT token to `/api/v1/vacancies/ai-search`.
3. FastAPI converts the query string into a vector embedding and runs a cosine distance match in PostgreSQL via pgvector.
4. The top matching results are returned as a JSON payload and rendered in the Angular grid.

7.3. Use Case UC-03: Cloud Sync of Filter Presets

- **Primary Actor:** Authenticated User
- **Preconditions:** User has configured specific search filters (e.g., Source: LinkedIn + Remote + Salary > \$100k).

Main Success Scenario

1. User clicks "Save Preset" in the web filter drawer.
2. User inputs a name (e.g., "Remote Senior Frontend").
3. Angular app issues a POST request to `/api/v1/users/me/presets`.
4. Backend verifies the JWT token and writes the preset configuration to PostgreSQL linked to the user ID.
5. The preset becomes immediately available on any web browser or desktop app where the user is logged in.